

Cheap Decomposition, Expensive Execution: Quantifying the Planning Quality Gap in Multi-Turn Tool-Agent Tasks

Bo-Yu Chen

boyu.chen@my.utsa.edu

Abstract

Multi-turn tool-agent tasks require both planning (what sub-tasks to perform) and execution (how to perform them). We ask: can a lightweight pre-planner improve agent reliability on τ -bench, a customer-service benchmark where state-of-the-art agents fail on over 40% of tasks? We define *pre-execution planning* as parsing a compound user request into typed sub-goal triples before any tool call, and test five conditions: no planning (baseline), rule-based, automated (Claude Haiku 4.5, \$0.0007/call), same-model (gpt-4o as both planner and executor), and oracle (gold sub-goals). On 22 complex retail tasks across 3 seeds, oracle planning improves mean pass¹ by 22.7 percentage points (OR=3.5, directionally consistent across all seeds), but both automated planners gain 0–1.5 pp ($p > 0.8$). A same-model ablation rules out model capability as the driver: gpt-4o as decomposer performs identically to Claude Haiku 4.5 ($p = 1.0$). A majority-vote robustness check (pass if $\geq 2/3$ seeds succeed, $n = 22$ independent tasks) confirms the direction of improvement but is underpowered to detect significance ($p = 0.75$); we report effect sizes throughout. Pre-execution planning has high headroom when sub-goal quality is sufficient, but automated decomposers, regardless of model size, capture only 50% of the necessary sub-goal structure from the first utterance. We release code and annotations.¹

1 Introduction

Function-calling agents can now handle multi-turn customer-service conversations, but they remain unreliable. On τ -bench [Yao et al., 2024], a benchmark of realistic retail and airline support tasks with tool use and policy constraints, state-of-the-art agents pass

individual tasks at roughly 60% (pass¹) and succeed on the same task three times in a row at only 35% (pass³). This reliability gap is the central barrier to deployment.

In cost-aware deployment, the question is not “which model is best?” but “where can a cheaper model substitute without quality loss?” Two substitution points exist. First, a cheap executor with an expensive planner: quality is expected to degrade because execution in multi-turn tool-agent tasks requires policy compliance, error handling, and multi-step API orchestration. Second, a cheap planner with an expensive executor: untested on multi-turn agents. We test the second option.

We propose *pre-execution planning*: before the agent makes any tool call, a lightweight model parses the user request into an ordered sub-goal list. Each sub-goal is a triple of (action_type, entity, constraint) that specifies what operation to perform, on what object, under what conditions. The agent then executes sub-goals using its full capabilities through the standard function-calling loop.

Our main finding: oracle planning (gold sub-goals) improves pass¹ by 22.7 pp (OR=3.5), but automated planners, including a same-model ablation using the executor model itself, gain 0–1.5 pp. The 22.7 pp *planning quality gap* is diagnostically interpretable: both automated planners generate half the oracle’s sub-goals and cannot resolve entity ambiguity from the first utterance alone. Contributions: (1) a formal definition of pre-execution planning for multi-turn tool-agent tasks, (2) a 7-category failure taxonomy of 215 failed trajectories, and (3) a five-condition experiment isolating the planning ceiling, including a same-model ablation that rules out model capability as the primary driver.

¹Code: <https://github.com/drewOrc/tau-bench-decomposition>

2 Related Work

Two axes organize prior work: *when* decomposition occurs and *whether* the decomposer and executor share the same model.

Upfront decomposition, same model. Plan-and-Solve [Wang et al., 2023] and Least-to-Most [Zhou et al., 2023] decompose problems before solving, in single-turn, tool-free settings. Decomposed Prompting [Khot et al., 2023] routes sub-problems to specialized prompts, demonstrating that decomposition and execution need not share the same prompt or model. Our work ports this intuition to multi-turn tool-agent tasks and splits the two stages across models of different cost.

As-needed decomposition, same model. ReAct [Yao et al., 2023b] interleaves reasoning with tool actions, forming the default agent loop in τ -bench. ADaPT [Prasad et al., 2024] recursively decomposes on failure. Tree of Thoughts [Yao et al., 2023a] explores reasoning paths with backtracking. Reflexion [Shinn et al., 2023] stores verbal self-critiques across attempts. All use the same model for both stages and most incur multiplicative cost. Our decomposer is proactive and costs 0.08% of the executor.

Cost-aware model selection. FrugalGPT [Chen et al., 2023] cascades models for single-turn QA. HuggingGPT [Shen et al., 2023] routes sub-tasks to specialist *executors*; we use the same executor for all sub-goals with only the *planning* stage delegated. Our earlier work [Chen, 2026] applies this principle to single-turn classification; the present paper extends it to multi-turn planning.

τ -bench. Yao et al. [2024] introduce τ -bench with pass^k exposing brittleness invisible to pass^1 . No prior work tests upfront decomposition with a cheaper model on a multi-turn tool-agent benchmark.

3 Method

3.1 Pre-Execution Planning

We define *pre-execution planning* as parsing a compound user request into an ordered sub-goal list before any tool call. Each sub-goal is a triple of (action_type, entity, constraint). Action-type specifies which category of operation to perform: in τ -retail, the six types are MODIFY, EXCHANGE, RETURN,

CANCEL, ADDRESS, and INFO. Entity identification specifies *what* to operate on; the downstream mapping to database IDs is execution. Constraints encode decision rules or are null for unconditional actions.

Unlike Plan-and-Solve [Wang et al., 2023] and Least-to-Most [Zhou et al., 2023], which decompose reasoning problems without specifying operation types, our definition targets proactive, one-shot planning with typed sub-goals. We reserve *execution* for resolving sub-goals into tool calls: database ID discovery, API parameter selection, and policy compliance. Our planning step augments the agent’s existing ReAct loop [Yao et al., 2023b].

3.2 Five Planning Conditions

Baseline (no planning). The standard function-calling setup in τ -bench.

Rule-based. A regex-based decomposer extracts action verbs and entity patterns from the user’s first utterance. Cost: \$0. On the full 115-task retail set, 27.5% of tasks receive zero sub-goals and 61.4% receive only one.

Oracle. A human annotator produces gold sub-goal triples from task instructions and ground-truth actions. We construct oracle sub-goals for 22 complex tasks selected by two criteria: ≥ 7 ground-truth actions (top 19% of action-count distribution) and baseline pass rate below 3/3. This subset selection conditions on baseline performance, so absolute numbers are specific to these 22 tasks; relative comparisons across conditions remain valid.

Automated (tiny-LM). Claude Haiku 4.5 receives the user’s first utterance and a structured prompt requesting JSON sub-goals. Cost: $\sim \$0.0007/\text{call}$ (0.08% of executor cost).

Same-model (ablation). gpt-4o as both decomposer and executor, with the identical prompt and output format as the tiny-LM condition. Cost: $\sim \$0.001/\text{call}$. If the quality gap is model-driven, same-model should approach oracle; if information-limited, it should cluster with tiny-LM.

All five conditions share the same executor (gpt-4o, 2026 snapshot, temperature 0), user simulator (gpt-4o), and seeds (42, 43, 44). The decomposer output is injected into the agent’s system prompt at turn 0; all subsequent turns proceed through the standard τ -bench pipeline.

4 Failure Taxonomy

We annotate all 215 failed baseline trajectories (139 retail, 76 airline) using an LLM-as-judge pipeline

[Zheng et al., 2023]. A gpt-4o-mini judge classifies each failure into one of seven categories. A 10-sample human spot-check yields 80% agreement on failure category (8/10). Disagreements arise when the judge labels incomplete execution as `partial_resolve` where the human identifies an underlying `wrong_decision`. The spot-check identifies one `user_led astray` case (10%) where the judge reports 0%, confirming under-annotation.

Table 1: Failure taxonomy (LLM-as-judge, $n=215$).

Category	Retail	Airline
<code>wrong_decision</code>	48.2%	65.8%
<code>partial_resolve</code>	39.6%	26.3%
<code>wrong_argument</code>	8.6%	3.9%
<code>wrong_info</code>	2.9%	3.9%
<code>user_led astray</code>	0%	0%
<code>unknown</code>	0.7%	0%

Two categories dominate: `wrong_decision` and `partial_resolve` together account for 87.8% of retail and 92.1% of airline failures. We flag one known bias: the decomposition-relevant flag is set at $\sim 98\%$, likely inflated; the human spot-check suggests $\sim 80\%$. Our oracle experiment (§5.2) provides a stronger test of the planning hypothesis than the taxonomy alone.

5 Experiments and Results

5.1 Baseline Reproduction

We reproduce the τ -bench baseline with the 2026 gpt-4o snapshot: τ -retail $59.7\% \pm 10.3$ pp (paper: 61.2%), τ -airline $49.3\% \pm 4.2$ pp (paper: 35.2%), 3 seeds. Our retail mean falls within the paper’s range; airline is higher, likely reflecting model improvements.

5.2 Main Results

Table 2 presents the main experiment on 22 complex retail tasks. The oracle condition improves mean pass¹ by 22.7 pp over the baseline (OR=3.5 in pooled analysis; directionally consistent across all three seeds; see §5.3 for majority-vote robustness analysis). The tiny-LM gains only 1.5 pp ($p = 1.0$). The same-model ablation gains 0.0 pp ($p = 0.838$), ruling out model capability as the primary driver. Oracle pass³ reaches 7/22 (31.8%) vs. baseline 0/22 (0% by selection criterion).

Table 2: Five-condition comparison (22 complex retail tasks, 3 seeds). [†]Pooled $n=66$; within-task correlation inflates effective n . Majority-vote ($n=22$) yields $p=0.75$; see §5.3.

Condition	S42	S43	S44	Mean $\pm$ std	Δ
Baseline	50.0	45.5	9.1	34.8 $\pm$ 22.4	—
Rule-based	45.5	36.4	18.2	33.3 $\pm$ 13.9	−1.5
Oracle	72.7	54.5	45.5	57.6$\pm$13.9	+22.7[†]
Tiny-LM	31.8	36.4	40.9	36.4 $\pm$ 4.5	+1.5
Same-model	36.4	36.4	31.8	34.8 $\pm$ 2.6	+0.0

The planning quality gap, defined as the difference between oracle and same-model planning, is **22.7 pp** (oracle 57.6% vs. same-model 34.8%; same-model chosen as reference because it controls for model capability). The gap against tiny-LM is 21.2 pp.

5.3 Statistical Tests

We use McNemar’s test with continuity correction on paired binary outcomes (22 tasks $\times$ 3 seeds = 66 pooled observations).

The oracle shows asymmetric improvement in pooled analysis ($n=66$): 21 tasks flip fail-to-pass vs. 6 pass-to-fail (OR=3.5, $p=0.007$). Same-model vs. baseline: $b=12$, $c=12$, $p=0.838$, OR=1.0. Same-model vs. tiny-LM: $p=1.0$. Holm–Bonferroni correction: oracle vs. baseline survives ($\alpha/6=0.0083$); oracle vs. tiny-LM ($p=0.018$) does not ($\alpha/5=0.010$). Direction is consistent across all three seeds (net discordant: +5, +2, +8).

Independence caveat. Pooling treats 66 pairs as independent, but within-task outcomes share task instructions across seeds. A leave-one-seed-out analysis: without seed 44, $p=0.146$; the other two remain significant.

Majority-vote robustness. Aggregating per-task (pass if $\geq 2/3$ seeds succeed) yields $n=22$ independent observations. Oracle 50.0% vs. baseline 40.9%, +9.1 pp, $p=0.75$. The loss of significance reflects reduced power (10 vs. 27 discordant pairs), not a reversal: oracle maintains the highest majority-vote rate across all five conditions. We report effect sizes as primary evidence throughout.

5.4 Variance Stabilization

All planning conditions reduce cross-seed variance. The coefficient of variation drops monotonically: baseline 0.64, rule-based 0.42, oracle 0.24, tiny-LM 0.12, same-model 0.08 (an $8\times$ reduction). The same-model planner achieves the lowest CV despite no accuracy gain, suggesting that structural pre-planning

stabilizes agent behavior even when sub-goal quality is low.

5.5 Diagnosing the Quality Gap

Why does automated planning fail to capture oracle-level quality? Table 3 compares sub-goal characteristics.

Table 3: Sub-goal quality comparison.

Metric	Oracle	Tiny-LM	Same-model
Mean sub-goals/task	3.0	1.5	1.4
Distinct action types	6	1.4	6
Entities “(unclear)”	0%	18%	30%
Tasks $<0.5 \times$ oracle	0/22	10/22	15/22

The primary failure mode is *under-decomposition*: both automated planners generate $<50\%$ of oracle sub-goals (1.5 and 1.4 vs. 3.0). The same-model planner uses all 6 action types but still under-decomposes and marks 30% of entities as “(unclear).” For example, Task 23 requires three operations (exchange helmet, exchange luggage set, modify grill). Both automated planners produce a single sub-goal targeting only the helmet. The identical failure pattern across a $175\times$ cost difference rules out model capability as the primary driver; the bottleneck is information available in the first utterance.

Both decomposers add $<0.12\%$ of executor cost (\$0.0007–\$0.001/call vs. $\sim$ \$0.85/task). The bottleneck is decomposition *quality*, not cost.

6 Discussion

6.1 When Does Cheap-Plan-Expensive-Execute Work?

Oracle planning yields the largest gain (+22.7 pp, OR=3.5). Neither automated condition improves: the same-model ablation (+0.0 pp, discordant pairs 12:12) rules out model capability, since even the executor model cannot produce useful decompositions from the first utterance alone. All planning conditions reduce cross-seed variance (baseline CV=0.64 $\rightarrow$ same-model CV=0.08), suggesting that structural pre-planning stabilizes behavior even when sub-goal quality is low.

6.2 The Quality Gap as a Research Target

The 22.7 pp gap quantifies the improvement available if decomposer quality can be raised. Three shared failure modes suggest directions:

1. **Under-decomposition.** Mean 1.5 sub-goals vs. oracle’s 3.0; 45% of tasks receive $<50\%$ of oracle sub-goals.
 2. **Action-type collapse.** Automated planners default to 1–2 action types (mean 1.4) vs. oracle’s full range of 6.
 3. **Entity ambiguity.** 18–30% of entities marked unclear. The first utterance uses pronouns and partial descriptions.
- Multi-turn context, retrieval-augmented planning, or domain-specific schemas could address these.

6.3 Limitations

1. **Single benchmark.** Generalization to WebArena [Zhou et al., 2024] or SWE-bench [Jimenez et al., 2024] is untested.
2. **Single executor.** Weaker executors may benefit more; stronger executors (o1-class) may benefit less.
3. **No human-human IAA.** The LLM-as-judge has known biases; an 80% human-LLM spot-check (§4) provides partial validation.
4. **Small task subset.** Majority-vote robustness ($n=22$) confirms direction but lacks power ($p=0.75$); the oracle effect should be interpreted through effect size (OR=3.5) and cross-seed consistency.
5. **Single oracle annotator** with access to ground-truth actions. A second annotator would strengthen validity.

7 Conclusion

On τ -bench compound tasks, pre-execution planning has high headroom: oracle sub-goals improve pass¹ by 22.7 pp (OR=3.5, directionally consistent) and achieve pass³ of 31.8% (vs. 0% by selection criterion). A majority-vote robustness check ($n=22$) confirms the direction but is underpowered; the evidence rests on effect size and cross-seed consistency. Neither automated planner (Claude Haiku 4.5 nor gpt-4o) yields significant gain. The same-model ablation is our most precise finding: model capability alone does not close the 22.7 pp quality gap ($p=1.0$), suggesting the first utterance lacks sufficient information.

The decomposer adds $<0.12\%$ of executor cost. Cost is not the bottleneck; decomposition quality is. Closing the gap requires decomposers that operate on more than the first utterance: multi-turn context, retrieval-augmented planning, or domain-specific schemas. We release all code, oracle sub-goals, and failure annotations.

References

- Bo-Yu Chen. Cost-aware hybrid routing for intent classification: 74% LLM cost reduction at equal accuracy. *arXiv preprint*, 2026.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. In *Proceedings of ICML*, 2023.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *Proceedings of ICLR*, 2024.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of ICLR*, 2023.
- Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, Mohit Bansal, and Tushar Khot. ADaPT: As-needed decomposition and planning with language models. In *Proceedings of NAACL*, 2024.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Proceedings of NeurIPS*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Proceedings of NeurIPS*, 2023.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of ACL*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Proceedings of NeurIPS*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*, 2023b.
- Shunyu Yao, Ankit Shinde, Karthik Narasimhan, and Yuntao Bai. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Proceedings of NeurIPS*, 2023.
- Denny Zhou, Nathaniel Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *Proceedings of ICLR*, 2023.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *Proceedings of ICLR*, 2024.